

Deriving Scaling Laws from Scratch

Nikos Fourlis

April 2026

Overview

This report presents an empirical derivation of scaling laws for a small Greek-language language model. Multiple models of varying sizes were trained on varying amounts of data, and the resulting validation losses were used to fit power-law relationships between model performance, parameter count, training tokens, and compute (FLOPs).

Dataset

Source: alexliap/tinystories-gr — 2.14M Greek children’s stories translated from the TinyStories corpus.

Preprocessing:

- Only the greek_translation column was used; English text was never seen by the model
 - Text was tokenized and concatenated with `<|endoftext|>` separators
 - Flattened token stream was chunked into sequences of length 257 (256 input + 1 target)
 - Chunks were shuffled once with a fixed seed (`numpy.random.default_rng(42)`) and cached to disk
 - 90/10 train/val split applied after shuffling
-

Tokenizer

A BPE tokenizer was trained from scratch on the Greek corpus using the HuggingFace tokenizers library.

Parameter	Value
Algorithm	Byte-level BPE
Vocabulary size	16,000
Special tokens	<\ endoftext\ >, <\ pad\ >
Pre-tokenizer	ByteLevel (preserves case and whitespace)
Decoder	ByteLevel

Justification: Greek has rich morphology with many inflectional suffixes. A vocabulary of 16,000 subword tokens allows the tokenizer to capture common stems and suffixes without fragmenting too aggressively. Byte-level BPE was chosen to handle all Unicode characters natively without any unknown tokens.

Model Architecture

A decoder-only transformer (GPT-style) was implemented in PyTorch.

Component	Details
Attention	Causal multi-head self-attention (scaled_dot_product_attention)
Normalization	Pre-LayerNorm
FFN	Linear → GELU → Linear (4× expansion)
Positional encoding	Learned embeddings
Weight tying	Token embedding = LM head weights

Experiment Grid

Models were trained across 4 sizes and 3 token counts, giving 12 runs total.

Model	Layers	d_model	Heads	Parameters
Small	4	64	4	~1.2M
Medium	4	128	4	~2.9M
Large	6	256	8	~8.9M
XLarge	8	512	8	~33.5M

Training tokens: 5M, 20M, 80M

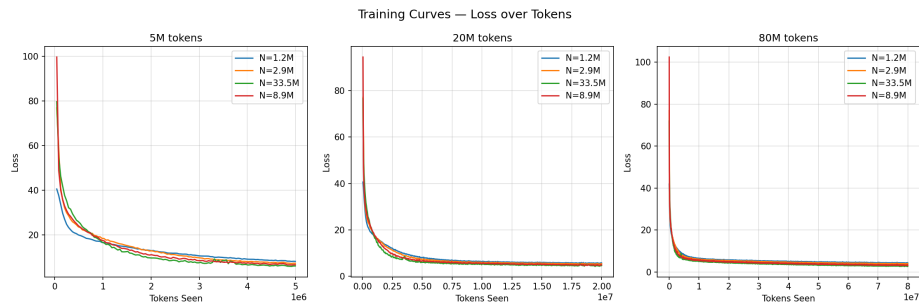
Other hyperparameters: batch size 32, AdamW optimizer, lr = 3e-4, no scheduler.

Results

Validation Loss Table

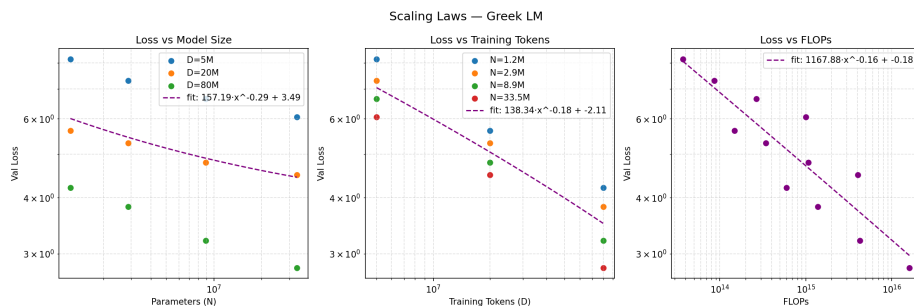
N (params)	D (tokens)	FLOPs	Val Loss
1.2M	5M	3.7e13	8.16
1.2M	20M	1.5e14	5.64
1.2M	80M	5.9e14	4.21
2.9M	5M	8.6e13	7.29
2.9M	20M	3.4e14	5.30
2.9M	80M	1.4e15	3.82
8.9M	5M	2.7e14	6.65
8.9M	20M	1.1e15	4.80
8.9M	80M	4.3e15	3.21
33.5M	5M	1.0e15	6.06
33.5M	20M	4.0e15	4.50
33.5M	80M	1.6e16	2.79

Training Curves



Training curves: loss over tokens for each model size

Scaling Law Plots



Scaling laws: loss vs N, D, and FLOPs on log-log axes

Scaling Law Analysis

A power law of the form $L(X) = a \cdot X^b + c$ was fit to the data for each axis using `scipy.optimize.curve_fit`.

Fitted Exponents

Axis	a	b	c
Parameters (N)	157.19	-0.295	3.492
Training Tokens (D)	138.34	-0.176	-2.110
FLOPs (C)	1167.88	-0.159	-0.175

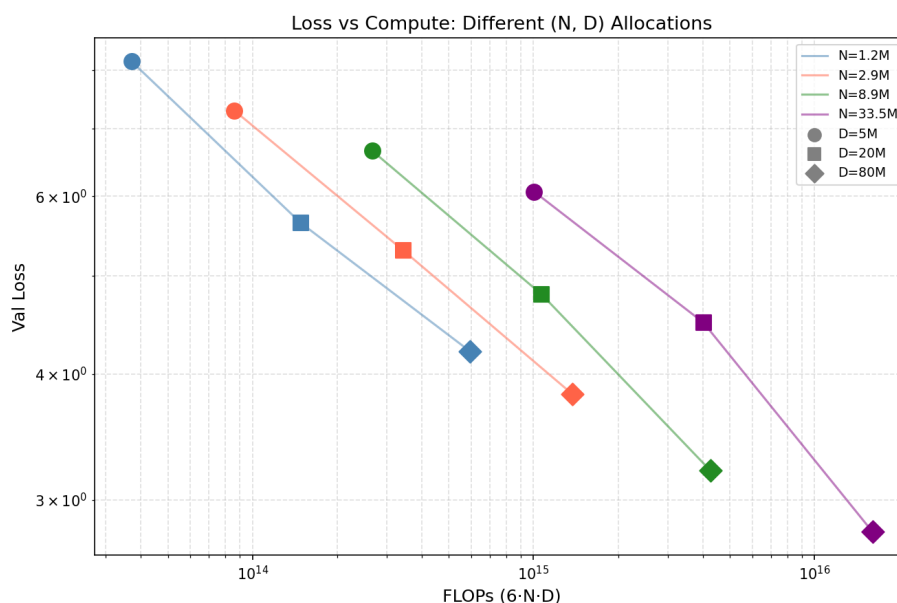
Comparison with Literature

Axis	This work	Kaplan et al. (2020)
N	-0.295	-0.076
D	-0.176	-0.095
FLOPs	-0.159	-0.050

Our exponents are steeper than Kaplan et al., meaning loss drops faster per unit of scale. This is expected given the small scale of our experiments — in the low-data, low-parameter regime, each additional token or parameter has a larger marginal effect. The trend saturates at scale, which is what Kaplan observed.

Compute-Optimal Allocation

The following plot shows all 12 runs on a single loss vs FLOPs chart. Each color represents a model size and each marker shape a token count. Lines connect runs of the same model trained on increasing amounts of data.



Loss vs Compute: Different (N, D) Allocations

For a given FLOP budget, runs that use a smaller model with more data consistently achieve lower loss than larger models trained on less data. For example, the 2.9M model trained on 80M tokens (1.4e15 FLOPs) outperforms the 33.5M model trained on 5M tokens (1.0e15 FLOPs), despite using similar compute. This is consistent with the Chinchilla finding that models are typically undertrained — allocating more tokens relative to parameters yields better performance per FLOP.

Limitations

- **No learning rate schedule:** A constant $lr=3e-4$ was used. A cosine decay schedule would likely improve final validation loss, especially for longer runs.
- **Small scale:** All runs are far below the scale of Kaplan or Chinchilla. Exponents may not generalize.
- **Training curves:** Per-step loss was logged every 5,000 tokens for each run.
- **Single seed:** Each configuration was run once. Variance across seeds is unknown.
- **Dataset quality:** The dataset is machine-translated Greek, which may introduce artifacts that affect loss measurements.

References

- Kaplan et al. (2020) — *Scaling Laws for Neural Language Models*
- Hoffmann et al. (2022) — *Training Compute-Optimal Large Language Models (Chinchilla)*
- Sardana & Frankle (2023) — *Beyond Chinchilla-Optimal*